

Time Complexity - Risk-Aware Shortest Paths

1. Time Complexity of `getBellmanFordPath`

Let:

- **V** = number of vertices
- **E** = number of edges

The method follows the standard Bellman-Ford structure, with extra work for negative-cycle detection and reconstruction.

1.1 Initialization

Iterates over all vertices to set initial distances.

- **$O(V)$**

1.2 Relaxation Phase

Outer loop runs **V - 1** times.

Inner loop scans all **E** edges each time.

- **Dominant term: $O(V \times E)$**

1.3 Negative-Cycle Detection

A single pass over all edges.

- **$O(E)$**

1.4 Negative-Cycle Reconstruction (if triggered)

- Backtracking through predecessors: **$O(V)$**
- Building the cycle: **$O(V)$**
- For each cycle edge, searching through all edges: **$O(V \times E)$**
- **Worst-case: $O(V \times E)$**

1.5 Normal Path Reconstruction

Backtracks from destination to origin, at most V vertices.

- $O(V)$

Overall Complexity of `getBellmanFordPath`

- $O(V \times E)$ (dominant term)
-

2. Time Complexity of `printBellmanFordPath`

This method only iterates over the precomputed results.

2.1 With a Negative Cycle

Iterates through cycle vertices/edges, at most V .

- $O(V)$

2.2 With a Standard Path

Iterates through the path, also at most V .

- $O(V)$
-

Final Summary

Method	Time Complexity
<code>getBellmanFordPath</code>	$O(V \times E)$
<code>printBellmanFordPath</code>	$O(V)$